

SEMI-AUTONOMOUS RED TEAMING FRAMEWORK

Backend Algorithms & Formulae Reference

The scoring, planning, policy, parsing, execution and cryptography mathematics of the assessment engine, stated precisely

Contents

Contents	1
1 Scope, Notation & Conventions	3
1.1 Bounding convention	3
1.2 Symbol glossary	3
2 Finding Scoring (backend/modules/analyzer.py)	3
2.1 Severity normalisation and score	3
2.2 Drivers and their bounds	3
2.3 Risk score	3
2.4 Priority score	3
2.5 Scoring algorithm	3
2.6 Worked example	4
3 Deduplication & Evidence Merging	4
3.1 Fingerprint	4
3.2 Merge operator	4
3.3 Pseudocode	4
4 Finding Extraction — Deterministic Per-tool Parsers	4
4.1 Common preprocessing	4
4.2 nmap — live hosts and open services	4
4.3 nuclei — template matches and severity folding	5
4.4 curl — security-header audit, banners, cookies	5
4.5 whatweb — technology fingerprints	5
4.6 sslscan — deprecated protocol detection	5
4.7 Normalisation guards	5
5 Provider Analysis & Token Budgeting	5
5.1 Budget allocation	5
6 Assessment Planning (backend/modules/planner.py)	5
6.1 Target classification and endpoint derivation	5
6.2 Default host plan	5

6.3 Subnet discovery and sweep sizing	5
6.4 Plan sources and policy filtering	6
7 Policy Engine & Scope Validation (policy_engine.py)	6
7.1 Design: allowlist over blocklist	6
7.2 Argument grammar	6
7.3 Host normalisation	6
7.4 Scope membership	7
7.5 Resolver policy	7
7.6 Decision procedure	7
8 Engagement Letter Parsing (engagement_parser.py)	7
8.1 Pipeline	7
8.2 Cleaning and furniture removal	7
8.3 Target-table extraction and the fallback scan	7
8.4 Restriction extraction — set algebra	7
8.5 Objectives and ruled-out lists	7
9 Command Execution & Assessment Lifecycle (executor.py, main.py)	7
9.1 Streaming, caps and truncation	7
9.2 Live terminal registry	7
9.3 Timeout and process-group kill	8
9.4 Retry, staleness and uniqueness	8
9.5 Assessment status automaton	8
10 Credential Encryption & Proxy Composition (secret_store.py)	8
10.1 Fernet envelope	8
10.2 Key hierarchy	8
10.3 Proxy environment composition	8
11 Tunable Constants & Bounds — Summary	8

1 Scope, Notation & Conventions

1.1 Bounding convention

1.2 Symbol glossary

σ	severity label, $\sigma \in \{\text{Low, Medium, High, Critical}\}$
$S(\sigma)$	severity score S : Low 25, Medium 50, High 75, Critical 100
$\rho(\sigma)$	severity rank ρ : Low 1, Medium 2, High 3, Critical 4
e, i, x	exploitability, impact, exposure — each in $[1, 5]$ (default 3)
c	confidence score, $c \in [0, 100]$ (default 70)
a	asset criticality, $a \in [0, 100]$ (default 70)
R	risk score (dimensionless product, $[1, 125]$)
P	priority score ($[15, 100]$)
A, D	A = enabled plan-step indices; D = indices already executed
T	set of tools known to the framework (8 tools)
τ	execution timeout, 360 s
N	a CIDR network; $ N $ = its address count
$\text{round}(\cdot)$	Python round — banker's rounding (half rounds to even)
$x \parallel y$	string concatenation
$\text{SHA-256}(\cdot)[0:24]$	full SHA-256 digest, kept as the first 24 hex digits

2 Finding Scoring (backend/modules/analyzer.py)

2.1 Severity normalisation and score

Severity σ	Score $S(\sigma)$	Rank $\rho(\sigma)$	Meaning
Low	25	1	Minor hardening gap or information exposure
Medium	50	2	Real but bounded weakness
High	75	3	Likely-exploitable weakness
Critical	100	4	Direct, high-impact compromise

2.2 Drivers and their bounds

Driver	Symbol	Range	Default	Persisted
Exploitability	e	1 – 5	3	exploitability
Impact	i	1 – 5	3	impact
Exposure	x	1 – 5	3	exposure
Confidence	c	0 – 100	70	confidence_score
Asset criticality	a	0 – 100	70	asset_criticality

2.3 Risk score

2.4 Priority score

2.5 Scoring algorithm

Algorithm 1 – score_finding(f): compute (σ , R, P, c)

```

input   : f  a finding dict with optional severity and driver fields
output  :  $\sigma$  normalised severity label
         R  risk score, P  priority score, c  confidence

1   $\sigma \leftarrow \text{normalize}(f.\text{severity})$            // §2.1; unknown  $\rightarrow$  Low
2   $e \leftarrow \text{clamp}(f.\text{exploitability}, \text{lo}=1, \text{hi}=5, \text{default}=3)$ 
3   $i \leftarrow \text{clamp}(f.\text{impact}, \text{lo}=1, \text{hi}=5, \text{default}=3)$ 
4   $x \leftarrow \text{clamp}(f.\text{exposure}, \text{lo}=1, \text{hi}=5, \text{default}=3)$ 
5   $c \leftarrow \text{clamp}(f.\text{confidence\_score}, \text{lo}=0, \text{hi}=100, \text{default}=70)$ 
6   $a \leftarrow \text{clamp}(f.\text{asset\_criticality}, \text{lo}=0, \text{hi}=100, \text{default}=70)$ 
7   $R \leftarrow e \cdot i \cdot x$                        // §2.3
8   $P \leftarrow \text{round}(0.40 \cdot S(\sigma) + 0.25 \cdot 20e + 0.20 \cdot a + 0.15 \cdot c)$  // §2.4
9  return ( $\sigma$ , R, P, c)

```

2.6 Worked example

3 Deduplication & Evidence Merging

3.1 Fingerprint

3.2 Merge operator

severity	$\sigma \leftarrow g.\sigma$ if $\rho(g.\sigma) > \rho(f.\sigma)$, else $f.\sigma$ (higher rank wins)
drivers	for each driver $\delta \in \{e, i, x, c, a\}$: $\delta \leftarrow \max(\text{clamp}(f.\delta), \text{clamp}(g.\delta))$
source_tools	$f.\text{tools} \leftarrow \text{sorted}(f.\text{tools} \cup g.\text{tools})$
evidence	$f.\text{evidence} \leftarrow f.\text{evidence} + "\n" + g.\text{evidence}$ if $g.\text{evidence}$ non-empty and not already present — then truncated to 20,000 chars
text fields	description, remediation, endpoint, parameter: the longer of the two is kept

3.3 Pseudocode

Algorithm 2 – combine(current, incoming) and the dedupe loop

```

combine(current, incoming):
  if rank(incoming. $\sigma$ ) > rank(current. $\sigma$ ):    current. $\sigma \leftarrow$  incoming. $\sigma$ 
  for  $\delta$  in (e, i, x, c, a):                     // element-wise strongest
    current. $\delta \leftarrow \max(\text{clamp}(\text{current}.\delta), \text{clamp}(\text{incoming}.\delta))$ 
  current.tools  $\leftarrow \text{sorted}(\text{current}.\text{tools} \cup \text{incoming}.\text{tools})$ 
  if incoming.evidence  $\neq$  "" and incoming.evidence  $\notin$  current.evidence:
    current.evidence  $\leftarrow (\text{current.evidence} + "\n" + \text{incoming.evidence})$ 
    truncated to L_text = 20 000 characters
  for text in (description, remediation, endpoint, parameter):
    if |incoming.text| > |current.text|:    current.text  $\leftarrow$  incoming.text
  return current

merge(raw findings):
  classes  $\leftarrow \{ \}$                                // key fp  $\rightarrow$  record
  for item in raw:
    n  $\leftarrow \text{normalize}(\text{item})$                      // clamp + cap field lengths
    key  $\leftarrow \text{fingerprint}(n)$                      // §3.1
    classes[key]  $\leftarrow \text{combine}(\text{classes}[\text{key}], n)$  if key in classes else n
  return [ score_finding(f) for f in classes.values() ]

```

4 Finding Extraction — Deterministic Per-tool Parsers

4.1 Common preprocessing

4.2 nmap — live hosts and open services

Evidence quality	Condition	Confidence c
Version identified	version field non-empty	95
Service only, confident	version empty, no '?'	70

Evidence quality	Condition	Confidence c
Tentative service match	service ends with '?'	55

4.3 nuclei — template matches and severity folding

Reported level	Folded severity σ	Drivers (e, i, x)	Confidence c
critical	Critical	(4, 5, 4)	80
high	High	(4, 4, 4)	80
medium	Medium	(3, 3, 4)	80
low	Low	(2, 2, 4)	80
info	Low	(1, 1, 3)	80

4.4 curl — security-header audit, banners, cookies

Required header	Severity	(e, i, x)	c	Rationale
content-security-policy	Medium	(3, 4, 4)	90	primary XSS / injection defence
strict-transport-security	Medium	(3, 3, 3)	90	prevents protocol downgrade
x-frame-options	Low	(2, 3, 4)	90	anti-clickjacking
x-content-type-options	Low	(2, 3, 4)	90	anti MIME-sniffing
referrer-policy	Low	(2, 2, 4)	90	leak of tokens in Referer

4.5 whatweb — technology fingerprints

4.6 sslscan — deprecated protocol detection

4.7 Normalisation guards

5 Provider Analysis & Token Budgeting

Bound	Symbol	Value
per-stream output cap	C_stream	120,000 characters
whole-batch budget	C_total	400,000 characters
max findings accepted	M	200
max text per text field	L_text	20,000 characters

5.1 Budget allocation

Algorithm 3 – budgeted prompt construction

```

budget ← C_total
for each scanner output o in plan order:
    avail ← max(budget, 0)
    o.stdout ← first min(C_stream, avail) chars of o.stdout
    budget ← budget - |o.stdout|
    avail ← max(budget, 0)
    o.stderr ← first min(C_stream, avail) chars of o.stderr
    budget ← budget - |o.stderr|
prompt ← instructions + JSON(truncated outputs) // model told output
                                              is evidence, not instructions

```

6 Assessment Planning (backend/modules/planner.py)

6.1 Target classification and endpoint derivation

6.2 Default host plan

6.3 Subnet discovery and sweep sizing

6.4 Plan sources and policy filtering

Source	When returned	Note
ai-filtered	model plan survived policy review	every surviving step carries capability + risk
default-unconfigured	provider key / URL / model missing	safe default plan used
default-provider-error	provider call or response failed	same safe default
default-policy-rejected	every model step failed policy review	default plan is used

Algorithm 4 – default_plan(target)

```

if target contains "/" and parses as IP network N:
    return subnet plan: // §6.3
    nmap -sn --max-rate 30 N
    nmap -sV --version-light --open --max-rate 30
    -p 22,25,53,80,110,143,443,445,3000,3306,3389,5432,
    5900,6379,8000,8080,8443 N
(host, port) ← urlparse(target) // §6.1
endpoint ← host:port if port else host
for step in DEFAULT_PLAN (7 fixed steps):
    target_arg ← host if step.tool ∈ {nmap, traceroute, dig, nslookup}
    else endpoint
    ports ← "-p " + port + " " if port and step.tool == nmap
    step.command ← step.command.format(target=target_arg, ports=ports)
return steps

```

7 Policy Engine & Scope Validation (policy_engine.py)

7.1 Design: allowlist over blocklist

7.2 Argument grammar

Flag class	Meaning	Example
bool_flags	self-contained switch, no value	nmap -sV, curl -I
value_flags	takes a value that is not a target	--max-rate 30
target_flags	takes a value that IS a target (scope-checked)	dig -q host, nuclei -u url
resolver_flags / prefix	value is a DNS resolver	dig @8.8.8.8
attached_patterns	self-contained flag regexes (-T4, +short, -p80)	nmap -T4
bundled_short_flags	POSIX bundle allowed; final char may take value	curl -sSL
resolver_positionals	positionals after this index are resolvers	nslookup name server

Algorithm 5 – scan_arguments(tokens, spec)

```

targets ← []; resolvers ← []; positionals ← []; pending ← none
for token in tokens after the executable:
    if pending:
        file token under targets or resolvers per pending kind; pending ← none
        continue
    if token starts with resolver prefix "@": resolvers ← token[1:]; continue
    if token is not a flag: positionals ← token; continue
    classify token:
        bool flag → nothing to do
        value / target / resolver flag → pending ← class (next token consumed)
        base=value forms: accepted when base is a known flag (--x=v)
        attached pattern match → accepted (self-contained)
        POSIX bundle expansion → accepted iff every char is bool, last may be value
        otherwise → REJECT (fail closed, report the token)
    if pending is still set: REJECT (a required value was missing)
    cutoff ← spec.resolver_positionals // e.g. nslookup = 1
    targets += positionals[:cutoff]; resolvers += positionals[cutoff:]
return (targets, resolvers)

```

7.3 Host normalisation

7.4 Scope membership

7.5 Resolver policy

7.6 Decision procedure

Algorithm 6 – validate_command(command, scopes, expected_tool)

```

1 if command empty → refuse
2 if any control character in command → refuse
3 tokens ← shlex.split(command) // shell-free; syntax error → refuse
4 if tokens[0] not in tool registry → refuse // executable not enabled
5 if expected_tool and tokens[0] ≠ expected_tool → refuse
6 (targets, resolvers) ← scan_arguments(tokens, spec) // Alg. 5
7 if scan rejected a token → refuse
8 if targets is empty → refuse // explicit target required
9 if any target not valid under Eq. 7.1 / 7.2 → refuse
10 if any resolver neither public nor in scope → refuse
11 allow: record capability and risk for the audit trail

```

8 Engagement Letter Parsing (engagement_parser.py)

8.1 Pipeline

8.2 Cleaning and furniture removal

8.3 Target-table extraction and the fallback scan

8.4 Restriction extraction — set algebra

allow-list “...only service discovery (nmap ...) and header inspection (curl, whatweb) are authorized against it” $\rightarrow R(t) \leftarrow R(t) \cup (T \setminus N)$ (everything not named becomes restricted)

deny-list “...nuclei ... must not be run against the DVWA lab” $\rightarrow R(t) \leftarrow R(t) \cup N$

8.5 Objectives and ruled-out lists

9 Command Execution & Assessment Lifecycle (executor.py, main.py)

9.1 Streaming, caps and truncation

Quantity	Symbol	Value
byte cap while draining	B_exec	800,000 bytes (= 4 × 200,000 chars)
char cap after decode	C_exec	200,000 characters
read chunk	—	65,536 bytes
truncation marker	—	“[stdout truncated]” / “[stderr truncated]”

Algorithm 7 – drain and truncate

```

retained ← 0; chunks ← [ ]
while pipe open:
    chunk ← read 65 536 bytes
    if retained < B_exec: // stop accumulating in time
        chunks += chunk limited to (B_exec - retained); retained += |chunk|
    append decoded chunk to live buffer (§9.2) if this is a live run
text ← decode(concat(chunks))
if |text| > C_exec: text ← text[:C_exec] + “\n[label truncated]”
return text

```

9.2 Live terminal registry

9.3 Timeout and process-group kill

9.4 Retry, staleness and uniqueness

9.5 Assessment status automaton

Status	Entered when
awaiting_approval	assessment created; after any step finishes while $D \subsetneq A$
running	an approved step is executing (start of <code>execute_step</code>)
ready_for_analysis	A non-empty and every enabled step has run: $D \supseteq A$
analyzed	analysis completed over all enabled steps
reported	HTML report generated from the analyzed findings

10 Credential Encryption & Proxy Composition (`secret_store.py`)

10.1 Fernet envelope

10.2 Key hierarchy

10.3 Proxy environment composition

11 Tunable Constants & Bounds — Summary

Constant	Value	Module	Guards
DEFAULT_ASSET_CRITICALITY	70	analyzer.py	default a when a target declares none
MAX_ANALYSIS_OUTPUT_CHARS	120,000	analyzer.py	per-stream output into the prompt
MAX_ANALYSIS_TOTAL_CHARS	400,000	analyzer.py	whole-batch prompt budget
MAX_FINDINGS	200	analyzer.py	findings accepted per assessment
MAX_FINDING_TEXT_CHARS	20,000	analyzer.py	evidence / description length
MAX_PLAN_STEPS	50	planner.py	steps in one plan
EXECUTION_TIMEOUT_SECONDS	360	executor.py	τ : subprocess deadline
MAX_OUTPUT_CHARS	200,000	executor.py	C_exec after decode
MAX_OUTPUT_BYTES	800,000	executor.py	B_exec retained while draining
LIVE_BUFFER_CHARS	200,000	executor.py	in-memory live tail window
MAX_REQUIREMENT_BYTES	5 MiB	main.py	uploaded requirement files
MAX_PLAN_FIELD_CHARS	100 / 4000 / 2000	main.py	tool / command / reason lengths
MAX_SUBNET_ADDRESSES	256	models.py	N for a CIDR target (env)
MAX_AUTHORIZED_SCOPES	50	models.py	scopes per target
MAX_RESTRICTED_TOOLS	20	models.py	restricted tools per target
MAX_BRIEF_ITEMS / MAX_BRIEF_CHARS	100 / 20,000	models.py	engagement-brief bound

Rounding: every `round()` call is Python's banker's rounding (half rounds to the nearest even integer). All four severity labels, three reachability drivers, confidence and criticality are persisted next to every finding so every score in this document can be re-derived by a reader from the raw evidence alone.